



Runtime

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v0.1.0

1. Introducción

1

1. Introducción

En la sección anterior, se produjo la generación de un pequeño programa pero, ¿qué sucedería si dicho programa fuera en Java o en otro lenguaje? Evidentemente no sería suficiente, ya que el programa generado:

```
final int x = 5;
final double result = add(123, sub(mul(4, 89), exp(div(x, 3), 8)));
log(result);
```

No representa un programa válido, en particular, porque no posee una función `main` definida, ni tampoco los métodos `add`, `sub`, `mul`, `exp`, `div` y `log`. Además, el programa por sí solo no es suficiente para generar un ejecutable, ya que requiere de la existencia de una máquina virtual (JVM, en particular), para lograr ejecutar dicho código en el sistema operativo objetivo.

Si fuera un programa más complejo, debería incluir librerías adicionales, o incluso también programas instalados en el entorno sobre el cual se ejecutará. A todas estas dependencias, incluyendo las modificaciones necesarias en el código generado que permitan que el programa final logre ejecutarse como es debido, se las agrupa dentro del concepto de **runtime**, es decir, de todo aquello que ocurre y que es necesario luego de la fase de compilación.

Parte de compilar un programa implica decidir qué partes deberán auto-generarse como parte de dicho proceso, y qué partes formarán parte del runtime destino del entorno sobre el que se instalará la aplicación final, es decir, el diseñador del lenguaje debe decidir qué parte del *runtime* será resuelta por el mismo compilador, y que por lo tanto formará parte del programa final, y qué parte formará aparte de las obligaciones del cliente que mantiene el entorno final.

En este ejemplo en particular, el código final en Java debería ser algo del estilo:

```
00 package ar.edu.itba.atlyc;
01
02 import java.lang.Math;
03
04 public final class Program {
05
06     private static double add(final double x, final double y) {
07         return x + y;
08     }
09
10     private static double sub(final double x, final double y) {
11         return x - y;
12     }
13
14     private static double mul(final double x, final double y) {
15         return x * y;
16     }
17
18     private static double div(final double x, final double y) {
19         return x / y;
20     }
21
22     private static double exp(final double x, final double y) {
23         return Math.pow(x, y);
24     }
25
26     private static void log(final double result) {
27         System.out.println("The result is: " + result);
28     }
29
30     public static void main(final String ... arguments) {
31         final int x = 5;
32         final double result = add(123, sub(mul(4, 89), exp(div(x, 3), 8)));
33         log(result);
34     }
35 }
```

Como se puede ver, el programa generado (resaltado en **color**), no basta para obtener un producto ejecutable. La inclusión de métodos adicionales sí permite que el programa sea válido, pero además, simplifica enormemente la generación de código, ya que evita inyectar directamente el código de cada método de forma *inline* mientras se produce la generación, lo cual en algunas situaciones podría incluso no ser posible (por ejemplo, si el código fuera mucho más extenso y además no fuera de un tipo compatible con *double*, no se podría incluir en la asignación de la línea 32).

En este caso, la generación de código debería incluir los fragmentos faltantes en el script para el nodo `program`. Si el código necesario abarca múltiples archivos, al igual que un proyecto medianamente complejo, entonces solo se generaría el programa principal, pero los demás archivos se deberían copiar desde una fuente que forme parte del compilador (como una carpeta de *assets*), hacia la carpeta final dónde se deposita la solución. Esta modalidad permite desarrollar libremente una librería que simplifique la generación de código, así como también facilita la mejora de la misma sin necesidad de reconstruir el compilador, ya que no forma parte del mismo.

Debido a que este ejemplo se desarrolla en Java, se deja en manos del cliente, la responsabilidad de disponer una instalación del *runtime*, es decir, de una JVM (*Java Virtual Machine*), con una versión que soporte el programa generado. Si el artefacto generado en este caso se denomina `Program.java`, entonces se debe compilar y ejecutar mediante:

```
javac -d . Program.java
java ar.edu.itba.atlyc.Program
```

Si el código generado fuera Python, entonces el cliente tendría la responsabilidad de instalar un entorno para dicho lenguaje. Lo mismo sucede si la salida fuera un documento DOT de Graphviz, aunque en ese caso no es un entorno completo, sino simplemente una aplicación disponible en la línea de comandos. Si fuera HTML+CSS+JavaScript, solo sería necesario un *browser* como Google Chrome o Mozilla Firefox, a menos que la ejecución del sitio web viole la política [Same Origin](#), en cuyo caso el cliente debería tener una instalación de algún servidor web. En caso de generar código en TypeScript, entonces se necesitaría una instalación de Node.js, o al menos de un instalador como NVM.

En general, la decisión de disponer parte del runtime en el compilador o en el cliente depende de la complejidad del lenguaje y de los requerimientos del mismo, pero también depende del nivel de **portabilidad** que se desea de la aplicación, ya que si gran parte del runtime se transporta en la solución, entonces mayor será la responsabilidad del diseñador del compilador de garantizar que los programas generados sigan siendo ejecutables en el entorno que el cliente posee.

Por otro lado, si la mayor parte del runtime se delega al cliente, entonces este tendrá que disponer un entorno más complejo, lo que implica que el mismo debe tomarse el trabajo de configurar las dependencias necesarias correctamente para que la solución se ejecute correctamente.